Roster Management (My Team(s) Tab)

Revision history — Jul 16, 2026: audited against the Team Stores Workspace prototype. Jul 28, 2026: tab renamed **My Team(s)**; roster-level Upload / Clear / Delete actions moved into the roster toolbar. Aug 25, 2026: duplicate-number policy re-scoped to the roster level. Aug 31, 2026: league structure relabelled to **divisions and teams**; rosters made optional and never a launch prerequisite; Contact Email made optional; team codes moved to backend-only. Aug 31, 2026 (later): this tab's structure component is **also used during League store creation** (see *Step 2 — Divisions & Teams (League Stores)*), and the manager-mapping surface is named **Settings → Distribution & Fulfillment** throughout, matching the Settings spec. Aug 31, 2026 (later still): **product visibility and assignment added to the League canvas and detail panels** (Epic 6) — team nodes carry product thumbnails, detail panels gain Setup / Products tabs, and a docked catalog drawer assigns by click or drag. Oct 13, 2026: **roster edits never mutate a placed order** — the order line snapshots personalization at purchase, number changes stay unrestricted, divergence is reported, and pending orders can be updated to match (Epics 2 and 4). All of the above are written inline below — this list is a changelog, not a set of overrides.

Executive Summary & Problem Statement

The Problem

An accurate roster drives pre-filled personalization, per-player order tracking, reminder sends, and freight sorting by participant. But requiring the rep to assemble one — names, numbers, emails, sizes, addresses — before a store is useful is friction that research with high-value reps found unacceptable. Reps do not reject player tracking; they reject the roster as an upfront obligation. The roster's outputs are valuable; the roster as a gate is not.

Jersey numbers compound the problem. Whether two players may share a number is not a league-wide truth — a 6U division may not care while a varsity team does — so a single league-wide switch forces the rep to pick the wrong answer for some of their teams. And a number conflict discovered at submit is a conflict discovered too late: the player has already committed to a number that was never available.

Product assignment had the same shape of problem from the other direction. Assignment was authored on the Products tab, one product at a time, answering "which teams get this product?" — but the question a rep running a league actually asks is the reverse: "what does this team get, and which of my teams is missing something?" Answering it meant opening every product in turn and assembling the picture by hand.

The Solution

A **My Team(s) tab** inside the Team Store platform where store managers (reps, team managers, league directors) build and manage team structure, and populate rosters **whenever they choose — before launch, during the ordering window, or not at all**. A roster is never required to launch, and a team may exist with no roster attached.

Rosters fill from either direction. **Front-load** paths (manual entry, CSV import, share-roster template) are unchanged and remain available. **Back-fill** arrives from the storefront: team-gear orders create captures in the **Roster Bank**, which the rep reviews and commits onto a team's roster. Both paths write the same record.

Duplicate-number policy is set **per roster**, defaulted in bulk from the league level, and enforced by making unavailable numbers unselectable rather than by rejecting them after the fact.

For League stores the tab also answers the **product coverage** question in place: teams on the canvas show what they carry, gaps are visibly amber rather than silently empty, and products are assigned to a team or a whole division from a docked catalog drawer without leaving the structure view (Epic 6).

Governing rule — permissive about the roster, immutable about the order

The roster is the rep's working list. An order line is a manufacturing instruction somebody has paid for. These are different kinds of object and they get different treatment.

- **Roster edits are never blocked by order state.** A rep may change a jersey number, a name on jersey, or a duplicate-number policy at any time, including on a player who has already ordered. Reps have been explicit that an over-restrictive system costs them more than the occasional data error, and the errors in this area are low-stakes and visible.
- **A roster edit never mutates a placed order.** Personalization is **snapshotted onto the order line at purchase** and is never re-read from the roster afterward (see *Order Management*). Changing a roster row changes the roster, not what the factory makes.
- **Divergence is reported, not prevented.** Where the two disagree, the tab says so plainly at the point the rep is already looking, and — while the order is still pending — offers to bring the order into line.
- **The cost of a mistake decides the treatment.** Two players sharing a number is a recoverable annoyance and is permitted. A submitted order silently changing its personalization is an unwearable garment and a chargeback, and is prevented by the snapshot rather than by a gate on the rep.

Object model

The My Team(s) tab manages four distinct things. The distinction between a **team** and its **roster** matters: a team can exist and function with no roster at all.

Entity	Applies to	Notes
Division	League stores only	The grouping tier — typically a city or club. Carries branding: name, color, sport. Contains teams. Optional; a league store may have none. Can hold **division-scoped product assignments** inherited by its teams (Epic 6).
Team	All stores	The unit everything else keys off: product assignment, shipping containers, fulfillment mapping, and the team code. A team sits **inside a division** or **stands alone** under the league. Every team must have a name.
Roster	All stores	The player list. **Optional** — a team may have no roster. Once created, a roster is attached to exactly one team, and one team holds at most one roster. There are no free-standing rosters.
Player record	All stores	A row on a roster. Schema below.

In a **Single-Team Locker** store there is no division tier and no grouping UI: the store is one team, and the My Team tab drops straight into that team's roster.

Each player record must contain the following defined schema

Only **First Name** and **Last Name** are required. A record may carry no contact channel at all — reps frequently do not hold parent contact details, and requiring one would reintroduce the upfront burden this tab exists to remove. Contact fields are validated for format when supplied and left empty otherwise.

Field	Required	Type / Validation	Notes

First Name	Required	String, max 50 characters	
Last Name	Required	String, max 50 characters	
Jersey Number	Optional	Integer, 00–99	Uniqueness depends on **that roster's** Allow Duplicate Numbers setting (Epic 4). Where duplicates are disallowed the number must be unique on that roster, taken numbers are locked at selection, and real-time conflict checking backstops any direct-entry path. Where duplicates are allowed, no uniqueness constraint applies. 2-digit numeric input. **Editing this field never alters an order already placed** — the ordered number is snapshotted on the order line (Epic 2).
Name on Jersey	Optional	String, max 15 characters	Converted to **UPPERCASE** dynamically. Snapshotted onto the order line at purchase, as Jersey Number is.

Contact Email	Optional	Valid email format, validated only when provided	Used for parent/guardian communication and share-link recipient pools where present. A record saves without one. Most commonly arrives from a storefront capture rather than from rep entry.
Contact Phone Number	Optional	String, validated to a 10-digit numeric format	Validated only when provided — a record saves without a phone number.
Player Shipping Address	Optional	Street, city, state, ZIP; validated against a map service on the backend	**Present only when the store's fulfillment is Direct to Home Delivery** (selected in Step 1 — Basics of the store-creation flow). **Optional** for the rep to enter — if left blank, the parent/player supplies it at storefront checkout and it **writes back onto the roster** (same pattern as Name on Jersey). Not collected at all for other fulfillment models.
Associated Emails	Optional	List of valid emails	Additional parent/guardian emails linked to the

			record; included in share-link recipient pools alongside the primary contact email where one exists.

Records with no contact channel. A record carrying neither an email nor a phone number is valid and fully functional for personalization, number assignment, product scoping, and freight sorting. Two behaviors degrade and must show it: **Share Team Store Link** offers copy-only (no send) for that player, and **Send Reminder** is disabled with a reason. Roster Bank captures usually supply a contact channel, so records committed from orders typically have one even when rep-entered records do not.

Functional Requirements & Core Workflows

User Stories

- As a rep, I want to launch a store without building a roster first, so that I can go live immediately and let the roster fill in from orders.
- As a rep running a league store, I want to organize teams into divisions that mirror my league's real structure, so that branding, product assignment, and order shipping all follow the same architecture.
- As a rep, I want to create a team with no roster and add one later — or never — so that structure and player data are independent decisions.
- As a rep or team manager, I want to add a player manually, so that I can quickly input a small team or make singular late adjustments.
- As a rep or team manager, I want to download a template, populate it, and re-upload it, so that I can quickly import a large roster of players.
- As a rep, I want to email the roster template to my team managers and share the team store link with selected players, so that roster-building and buying can start without me doing the data entry.
- As a rep, I want to add a player record without an email or phone number, so that a roster I can only partly fill is still worth building.
- As a rep or team manager, I want to edit any field of an existing player's record, so that I can correct spelling or update contact details.
- **As a rep, I want to fix a player's jersey number even after they have ordered, so that a typo is a two-second correction rather than something the system refuses.**
- **As a rep, I want to be told when a player's ordered number no longer matches their roster number, so that I find out from the roster rather than from a parent holding the wrong jersey.**

- **As a rep who catches a number mistake before anything ships, I want to push the correction onto the pending order, so that the right jersey gets made.**
- As a rep running a league, I want to set the duplicate-number rule per team rather than once for the whole league, so that a division that allows sharing and one that forbids it can coexist.
- As a rep, I want to set the rule for every team at once and then adjust the exceptions, so that per-roster control doesn't mean configuring twenty rosters by hand.
- **As a rep, I want to turn duplicates off on a roster that already has a shared number, so that tightening the rule going forward isn't blocked by history I can't change.**
- As a rep or parent picking a number, I want unavailable numbers to be unselectable, so that I never choose a number only to be told at submit that I can't have it.
- As a rep or team manager, I want to remove a player from the active roster, so that their jersey number is released back into the pool of available numbers.
- As a rep or team manager, I want to clear an entire roster or delete a roster outright — with a confirmation step — so that I can restart bad data entry or remove a roster created by mistake.
- As a rep running a league, I want to see at a glance which products each team carries, so that I can answer "what does this team get?" without opening every product.
- As a rep, I want teams with no products called out visually, so that a coverage gap is something I notice rather than something a parent discovers.
- As a rep, I want to assign a product to a team — or to an entire division at once — without leaving the structure view, so that fixing a gap is a drag rather than a detour.
- As a rep, I want to inspect and edit a product from here, so that spotting a wrong price on a team's strip does not mean navigating to another tab and finding it again.

Epic 0 — League structure: divisions & teams

In a League (Multi-Team Hub) store the My Teams tab organizes structure into **divisions** — e.g. a city or club, carrying one set of branding (name, color, sport) — which contain **teams** (the individual squads / age divisions). Teams may also **stand alone**, outside any division. Every team must have a name; a roster is optional on every team. This structure is shared platform-wide: product assignment (Products tab) and pending-order shipping containers (Orders tab) key off it.

- **Home view:** Given a League store, when the My Teams tab opens, then it shows a stats header (divisions · teams · players), a **canvas** of division headers and team nodes, a standalone-teams list, and **New Team** / **New Division** actions. Team nodes and division headers carry the product affordances specified in Epic 6.
- **Single-Team Locker:** Given a Single-Team store, then no division tier and no grouping UI is shown — the My Team tab drops straight into the team's player table.

- **Create / edit / delete:** divisions are created with a name and brand color and edited in a modal (including fulfillment-manager mode); teams can be added to a division or standalone, renamed, and deleted with confirmation. **Deleting a division keeps its teams** — they become standalone teams, dropping the division's branding, and each keeps its roster and players intact.
- **This component is also used during League store creation.** The same structure editor renders as **Step 2 of League store creation** — see *Step 2 — Divisions & Teams (League Stores)* — with **branding and the team code omitted from the detail panel** in that context. It is one component with two hosts, not two editors: behavior specified here applies in both, and structure built during creation is indistinguishable from structure built in this tab. Any change to this editor must be assessed against both surfaces.
  - **Product affordances are workspace-only.** The Epic 6 surfaces — thumbnail strips, count pills, the Products tab of the detail panel, and the catalog drawer — are **not rendered in the creation step**, because the store has no catalog until the Add Products step that follows. The creation-context detail panel shows Setup content only, with no segmented tabs.
- **Drill-in:** opening a division lists its teams; opening a team shows breadcrumbs and a branded banner (division color, team avatar) above the player table. In the workspace the detail panel is split into **Setup** and **Products** tabs (Epic 6). Each division and team carries a deterministic, human-readable **team code** (e.g. RT47A).
- **Team codes are a backend key.** Codes identify a division or team for order routing, container assignment, and reporting. They are **not entered by the rep** and are **not entered by the shopper** — the storefront resolves a shopper to a team with a team selector at arrival (see the Roster-Optional Architecture spec, Epic D). Codes remain visible in the workspace for reference and support, and are not surfaced at all during store creation.
- **Fulfillment mapping:** each division routes orders to the **league manager**, a **division manager**, or **individual team managers**. Mapping is configurable in **Settings → Distribution & Fulfillment** (the Order Fulfillment Mapping, specified in *Store General Settings*, Epic A3) and directly on the **division and team detail components**; both surfaces read and write the same state. Division and team views surface an **Order Fulfillment strip** showing the effective manager or a "Not assigned" state with a Manage link — unassigned managers gate pending-order submission (see the Order Management spec).

Epic 1 — Filling the roster

A team's roster can be filled by any combination of the paths below, at any point in the store lifecycle, plus **Roster Bank commits** (Epic 5). Front-loading and back-filling coexist on the same roster. The manual and CSV paths are **also available during League store creation** (*Step 2 — Divisions & Teams*, Epic C) for reps who prefer to configure everything upfront; records created there are ordinary player records.

Add manually (1-by-1). An **inline row** at the foot of the player table ("Add Player") containing all player data fields.

- Given the add row, when it opens, then it contains all player data fields from the schema above (the address field appears only for Direct-to-Home stores). Only First and Last Name must be completed.
- **Number entry respects the roster's policy:** the Jersey Number control follows Epic 4 — on a roster that disallows duplicates, already-assigned numbers are presented locked and unselectable; the inline error remains as a backstop for direct entry.
- Given a valid record, when I confirm the row (green check), then the record is added to the roster table instantly.

Roster data import (batch CSV).

- **On-page entry: Upload Roster** is a primary button in the roster view's toolbar, beside **Clear Roster**, **Delete Roster** (where offered), and the sort control.
- Given the **Upload Roster** modal, when it opens, then it offers a drag-and-drop / click-to-browse zone and a **"Download CSV template"** link — a structured .csv skeleton with headers matching the data model.
- **Per-row validation with skip semantics:** Given an uploaded file, when the parse engine validates rows, then errors are reported per row and cell (e.g. *"Row 12: Missing last name"*, *"Row 4: Duplicate jersey number 8 (also Row 2)"*); invalid rows are **skipped** ("n rows skipped — fix and re-upload") while valid rows import, with a preview table and valid/skipped counts before confirming.
- **Only missing required fields skip a row.** A row lacking an email or phone number imports normally. Email and phone are validated for **format only when a value is present** — a malformed email skips the row, an absent one does not.
- **Duplicate-number reporting on import is policy-aware:** a duplicate number is a **skip-worthy error only on rosters that disallow duplicates**. On a roster that allows them, duplicate numbers import without error.
- **Share Roster Template:** a standalone flow emails the blank roster template (prolook_roster_template.csv) to team managers so they can fill in player details and return it for import; the message is pre-drafted and signed by the logged-in rep. Manager email addresses come from the manager records in Settings, not from the roster, so this flow is unaffected by players having no contact details.

Epic 2 — Managing & editing player records

- **Edit:** Given a player row, when I open its **3-dot row menu** and choose Edit, then a **side drawer** opens pre-populated with the player's current details; the jersey-number control follows the roster's duplicate policy (Epic 4), with the player's own current number always selectable.

- **Delete:** a single player is removed from the row menu instantly; **bulk delete** — via row checkboxes and the selection bar — shows a confirmation modal before removing the selected players. Removed jersey numbers are freed back into the available pool and become selectable again immediately.
- **Clear Roster:** Given the roster view, when I click **Clear Roster**, then a confirmation modal states how many players will be removed and that the roster itself is kept; on confirm, all players are removed (numbers released) and a success toast confirms ("Roster cleared — n players removed"). The **roster entity survives** and remains attached to its team. Disabled when the roster is already empty.
- **Delete Roster:** Given a roster in a League store, when I click **Delete Roster**, then a confirmation modal states the roster and its n players will be permanently deleted; on confirm the **roster entity is removed** and its team survives with no roster attached, the view returns to the team detail, and a success toast confirms ("Roster "name" deleted"). Not offered on a Single-Team store's only roster — use Clear Roster there. Deleting a **team** is a separate action in Epic 0 and removes the team along with any roster it holds.
- **Neither action touches the Roster Bank.** Captures are not roster members — they live in the Bank until a rep commits them (Epic 5). Clear Roster and Delete Roster affect committed player records only; pending captures for that team remain in the Bank and are still available to commit. Confirmation-modal counts state committed players only, and where captures are pending the modal says so.
- **Neither action touches product assignments.** Clearing or deleting a roster leaves the team's assigned products (Epic 6) untouched — products belong to the team, not to its player list.
- **Share Team Store Link:** the row menu copies a per-player store invite link; selecting multiple players surfaces a bulk **Share Team Store Link** modal with recipients pre-filled from the selected players' contact + associated emails. Players with no email contribute no recipient and are listed as copy-only in the modal; where no selected player has an email, the modal offers the link for manual distribution rather than a send.
- **Team Gear column:** each row shows the player's team-gear completion ("n/m team gear items") and opens their gear drawer — ordered items vs. missing required gear, with Send Reminder (see the Order Management spec). **Send Reminder is disabled, with a reason, for a player with no contact channel.**
- **Sorting:** a toolbar sorts the table by Name (A–Z / Z–A) or Number (low→high / high→low), with a live player count.

Editing a player who has already ordered

Reps have been explicit that they would rather live with occasional bad data than be locked out of simple corrections. That preference governs the roster. It does not extend to orders, because a changed order is not a recoverable annoyance — it is a garment nobody can wear.

- **Personalization edits are never blocked by order state.** Given a player with one or more team-gear orders, when the rep edits **Jersey Number** or **Name on Jersey**, then the edit saves with no block, no confirmation modal, and no warning interstitial. A rep correcting a typo meets no friction whatsoever.
- **The edit does not reach the order.** Personalization is **snapshotted onto the order line at purchase** and is never re-read from the roster afterward (owned by *Order Management*). Changing the roster changes the roster; the factory still makes what was bought.
- **Divergence is reported where the rep already looks.** Given a player whose ordered personalization differs from their current roster values, when the **Team Gear** column and the player's gear drawer render, then the affected item states both — *ordered as #12 · roster says #7*. This is passive information, not an error state and not a task: the rep may have changed the number precisely because the player's situation changed, and the system has no way to know which value is "right".
- **While the order is still pending, offer to fix it.** Given an affected order that has **not yet been submitted to the factory**, when the divergence line renders, then it carries a single action — *update the order to #7* — which rewrites that order line's snapshot to the current roster values. The pending window already exists for MOQ batching (*Order Management*, Epic B), so this reuses a genuine window rather than inventing one, and it covers the common case of a typo caught before production.
- **Once the order is submitted, the action is withdrawn and the information stays.** Given a submitted order, then the divergence line renders without an action. The rep is told what will arrive so they can warn the parent or arrange a replacement; nothing in the product pretends it can still be changed.
- **Updating the order is per order line, never bulk.** Each affected line is updated on its own action. A bulk "sync roster to orders" control is deliberately not offered — it would make a single mis-click capable of rewriting many paid orders at once, which is the one outcome this whole section exists to prevent.
- **Divergence never gates anything.** It does not block submission, does not affect the launch gate, does not appear as a Home attention card, and does not change the player's team-gear completion count — the item was ordered, and it stays ordered.
  - **Why no Home card:** the condition would clear when the order is submitted, which is exactly when the rep loses the ability to act on it. A card vanishing at that moment reads as resolution and would be actively misleading.

Epic 3 — Direct-to-Home Delivery: player shipping address

Applies **only when the store's fulfillment is Direct to Home Delivery** (chosen in Step 1 — Basics). In this scenario each player needs a shipping destination, but the rep is not required to know it up front.

- **Conditional field & column:** Given the store uses **Direct to Home Delivery**, when the My Team(s) tab renders, then the add/edit form exposes a **Player Shipping Address** input and the roster table shows an additional *Player Address* column. For all other fulfillment models the field and column are **not shown**.
- **Optional entry:** Given the address field, then it is **optional** for the rep — the record can be saved without it (mirrors Name on Jersey).
- **Map-service validation:** Given a rep enters an address, when the record is saved, then the address is **validated against a map service on the backend** before it is accepted — an inline "Verifying address with map service..." state shows during the check, and a failed check offers a retry-or-leave-blank error.
- **Checkout write-back:** Given a player record already exists and the rep left the address blank, when the parent/player enters their shipping address **at storefront checkout**, then that address **writes back onto the roster** and is reflected in the *Player Address* column with a "From checkout" badge; blank addresses show an "Awaiting checkout" placeholder.
- **Write-back vs. capture:** write-back completes a record that already exists. Where **no** matching player record exists, the order does not write back — it creates a **Roster Bank capture** instead (Epic 5). These are the two directions of the same plumbing and must not both fire for one order.
- **Write-back is the one order → roster path, and it is not reversible.** Checkout writes an address onto the roster; nothing in this tab writes an address back onto a placed order. A rep correcting a shipping address after an order exists is correcting the roster for next time — the existing order ships to the address it was placed with, and the divergence treatment in Epic 2 applies to the *Player Address* column in the same way it applies to personalization.

Epic 4 — Duplicate-number policy & number selection

Whether two players may share a jersey number is a **per-roster** decision, not a league-wide one.

Scope

- **The roster owns the policy.** Each roster carries its own **Allow Duplicate Numbers** setting, set in the roster view and governing that roster only. A roster's own value is what validation reads — always.
- **The league-level control is a bulk default, not an enforcement layer.** The **Allow Duplicate Numbers** control in Settings → Team & Roster (Roster Rules card) behaves as a **select-all / bulk apply**: setting it at the league level **cascades through every division to every team's roster in the store**. Any roster may then override it, and the league-level control never overrides a roster afterward.

- **The same league-level value can be set before the store exists.** *Step 1 — Basics*, Epic D1 surfaces the Allow Duplicate Numbers control during store creation, writing the **same store-level field** the Settings card reads. It is one value with two entry points, not a creation-time copy.
- **New rosters** inherit the league-level default at creation, then diverge freely. This includes rosters created during League store creation (*Step 2 — Divisions & Teams*, Epic C), which read the value set in Step 1.
- **Mixed states are normal and must be legible.** Where rosters disagree with the league-level default, the Settings control reports a mixed state rather than implying uniformity, and each roster view states the policy currently in force — so a rep never has to open Settings to understand why a number was accepted or rejected.

When duplicates are NOT allowed — lock, don't flag

- **Taken numbers are locked and unselectable.** Given a roster that disallows duplicates, when the rep or a shopper reaches jersey-number entry, then numbers already assigned on that roster render **disabled and non-selectable**. The conflict is prevented at selection, not reported at submit.
- **Uncommitted Roster Bank captures hold their numbers.** The taken-number set for a roster is the union of its **committed player records** and the **pending captures banked against that team**. A number claimed by a capture nobody has committed yet is unavailable — otherwise two buyers could take the same number in the gap between order and commit.
- **Every number-entry surface obeys this:** the roster add row, the edit drawer, the storefront PDP jersey-number entry, and checkout. A number that cannot be had must never appear selectable anywhere.
- **Number grid at selection.** Number entry presents the roster's full number range as a grid with taken numbers greyed out, so the picker chooses from what is genuinely available instead of guessing. The player's own current number stays selectable when editing their record.
- **The inline error becomes a backstop.** *"Number [X] is already assigned to [Player Name]"* still fires, but only for paths that bypass the picker — CSV import and direct field entry. It is no longer the primary mechanism.
- **Releasing a number** through player delete, bulk delete, or Clear Roster returns it to selectable immediately. A number held by a **pending capture** is released when that capture is dismissed from the Roster Bank, not by clearing the roster.

When duplicates ARE allowed

- Numbers remain fully selectable and no conflict error fires. Pending captures impose no constraint.

- A number already in use is labelled **"duplicate"** rather than **"taken"** — on a roster where sharing is deliberate, "taken" reads as a blocker and misleads the picker.

Turning duplicates off on a roster that already has one

Tightening the policy is a forward-looking decision, and the rep making it usually cannot undo the history that preceded it — two players may have ordered the same number already, and a jersey in production cannot be un-numbered.

- **The switch is never blocked.** Given a roster carrying one or more shared numbers, when the rep turns **Allow Duplicate Numbers** off, then the setting saves. No validation sweep runs, no conflict list is presented, and the rep is not asked to resolve anything first.
- **Existing duplicates are grandfathered.** Given a shared number assigned while duplicates were allowed, when the policy is turned off, then those assignments **stand**. No record is altered, no number is released, and no order is affected.
- **The new rule governs new selections only.** Given the policy is off, when any number-entry surface renders, then the taken-number set and the locking behavior above apply from that point on. A grandfathered number counts as taken, so no third player can join it.
- **The roster reports the exception rather than asserting a rule that is currently false.** Given a roster carrying grandfathered duplicates, when the roster view renders, then it states the condition and its count — *duplicates off · 2 players share #7, assigned before the rule changed* — with the affected rows identifiable. This is the same three-state honesty the product already applies to fundraising disabled-with-exemptions (*Store General Settings*, Epic A4) and to the Roster Rules mixed state.
- **The exception clears itself.** Given a grandfathered duplicate, when one of the sharing players is deleted, moved, or renumbered, then the condition resolves with no further action. Nothing prompts the rep to resolve it, and nothing is gated while it stands.
- **Turning duplicates back on** clears the exception reporting and restores ordinary shared-number behavior.

Epic 5 — The Roster Bank in the My Team(s) tab

Team-gear orders create **captures** — potential roster entries that wait for a rep to commit them. The Bank's capture, pre-grouping, and commit behavior is specified in the **Roster-Optional Architecture (Open Access Stores)** spec, Epic C. This section covers only where it lives in this tab and how it interacts with the surfaces above.

- **Two entry points, one queue.** The Roster Bank is available **per team** — from the team detail view, showing that team's pending captures — and as a **store-wide view** from the My Team(s) home, showing all pending captures with a team column and filters. Both read the same records; committing from either has the same effect.
- **Counts surface where the rep already looks.** Team and division cards, and the store-wide stats header, show a pending-capture count alongside the player count so a rep sees

unreviewed captures without opening the Bank.

- **Committing writes a normal player record.** A committed capture becomes an ordinary row on that team's roster and is indistinguishable from a manually entered one thereafter. If the team has no roster yet, committing the first capture creates one.
- **Captures are not players.** They do not appear in the player table, are not counted in player counts, and are not affected by Clear Roster or Delete Roster (Epic 2) — but they **do** hold jersey numbers (Epic 4) and **do** count toward team-gear completion tracking.
- **A capture commits at the values it was captured with.** Committing writes the personalization the buyer entered at checkout, so a freshly committed record never diverges from the order that created it. Divergence (Epic 2) can only arise from a rep edit afterward.
- **Not present during store creation.** The Bank has nothing to hold before a storefront exists, so it is not surfaced in the League creation step (*Step 2 — Divisions & Teams*, Epic C).

Epic 6 — Product visibility & assignment (League stores)

League stores only. A Single-Team Locker has one team and no coverage question — the store's catalog is that team's catalog — so none of the surfaces below render there. They are also absent during store creation (Epic 0), where no catalog exists yet.

Assignment itself is **not redefined here**. The catalog record, its team setting, its pricing, and the team-code bindings it carries are owned by the **Products tab** spec. This Epic specifies a second, structure-first surface onto that same state: the Products tab answers "which teams get this product?", and this tab answers "what does this team get, and who is missing something?" Both read and write the same assignments, and neither is authoritative over the other.

On the canvas

- **Team product strip:** Given a League store, when the canvas renders, then each **team node** carries a **strip of product thumbnails** showing what that team currently carries.
- **Gaps are amber, not empty:** Given a team with no assigned products, when its node renders, then the strip shows an **amber "No products yet"** state rather than blank space. A team carrying nothing is a condition to notice, not an absence to overlook.
- **Per-node add:** each team node carries a **+** that opens the catalog drawer scoped to that team (see *Add from Catalog* below).
- **Division count pill:** Given a division header, when it renders, then it shows a **product count pill** for the division — the number of products reaching its teams, whether assigned division-wide or team-by-team.

Inspecting a product without leaving the tab

- **Hover card:** Given a product thumbnail on a team node, when the rep hovers it, then a **floating product card** appears carrying the same content as the **Products-tab product card** — thumbnail, product name, design ID, team setting, assigned teams, and Youth / Adult prices. It is the same card component, not a lookalike.
- **Full detail in a modal:** Given a product thumbnail, or a **product row title** in the detail panel, when the rep clicks it, then the **full product detail page opens in a modal** over the tab. **Editing and saving work from there** and write to the catalog record directly — a wrong price spotted on a team's strip is corrected without navigating to the Products tab and finding the product again.
- Closing the modal returns the rep to the canvas or panel they opened it from, with any edit reflected immediately in the strip and cards.

Detail panel — Setup / Products tabs

The division and team detail panels are split by a **segmented control**: **Setup** and **Products**.

- **Setup** holds the existing panel content — roster, the Order Fulfillment strip, rename and delete.
- **Team detail → Team Products tab.** Given a team, when the Products tab opens, then it lists the products assigned to that team, with rows sourced from a division-wide assignment marked **"via division"** so the rep can tell inherited coverage from coverage assigned to this team specifically. The tab carries an **Add Products** split menu — **Add New Product** (hands off to the existing add-product flow) and **Add from Catalog** (opens the drawer) — and a **Roster** button jumping to that team's player table.
- **Division detail → Division Products tab.** Given a division, when the Products tab opens, then it shows the **division-wide products list** plus a **per-team coverage table** — every team in the division and what it carries. Teams with no coverage show an **amber "+ Assign products"** action inline, so the fix is offered at the point the gap is visible. The tab carries **Add Products** and **Add Team**.
- **"+ Add Product"** also sits in the tab header, so the action is reachable without opening the split menu.

Add from Catalog — the docked drawer

- **Docks, does not navigate:** Given **Add from Catalog** from any entry point, when it opens, then the catalog **docks as a right-hand drawer** beside the canvas or panel. The rep never leaves the structure view, so the thing they are assigning to stays visible while they choose.
- **Finding a product:** the drawer offers **search** and three filters — **All**, **Unassigned** (products on no team), and **Not on [X]** (products absent from the launching team or division, named explicitly).

- **Click to assign:** Given a product in the drawer, when the rep clicks **Assign**, then it is assigned to the **launching scope** — the team or division whose **+** or Add Products opened the drawer.
- **Drag to assign anywhere:** Given the **grip** on a drawer product, when the rep drags it onto **any team node**, then it is assigned to that team — not only the launching scope. Dragging onto a **division header** creates a **division-wide assignment covering all of its teams**, which those teams surface as **"via division"** rows.
- **Immediate feedback:** an assignment updates the target's thumbnail strip, the division count pill, and the coverage table without a reload, and clears any amber gap state it resolves.
- Division-wide assignment extends forward. A product assigned to a division applies to every team

  in that division, including teams added to the division after the assignment was made. No

  re-assignment step is required when a team is added, and the assignment is not recalculated or

  confirmed at the point a team joins.

Epic 7 — Move a player to another team

- Given a player row in a roster, when the rep opens the row menu and chooses **Move to team**,

  then a destination picker opens listing every other team in the store. In League stores the list

  is grouped by division and is searchable.
- Given selection mode with one or more players selected, when the rep chooses **Move to team**,

  then the same picker opens and applies to the whole selection.
- Given a destination, when the rep confirms, then each player leaves the origin roster and joins

  the destination roster. The player's jersey number is released on the origin roster and becomes

  selectable there immediately.

- Given a player's jersey number, when that number is already taken on the destination roster or is

  disallowed by the destination roster's duplicate-number policy, then the rep is prompted to

  assign a different number or to move the player without one. The taken-number set at the

  destination follows Epic 4 — committed player records and pending Roster Bank captures both

  count.
- Given a player with one or more team-gear orders, when a move is attempted, then the move is

  refused, the control is disabled, and the reason is stated. In a bulk move, blocked players

  remain on the origin roster and the result names each one.
- Product assignment does not travel with the player. A moved player inherits the destination

  team's assigned products (Epic 6) and loses access to products assigned only to the origin team.
- Moving the last player off a roster leaves the roster entity in place, as Clear Roster does

  (Epic 2). The roster is not deleted by becoming empty.

OPEN — the order-state block above should be reconsidered. It is now the only hard block in this tab, and it sits against the governing rule at the top of this page: every other roster edit is permitted and its consequence reported. Under the snapshot model there is no data reason to refuse a move — the existing order keeps its own personalization and is unaffected. There is, however, a real **operational** reason: that order is routed to the **origin team's shipping container**, so it will ship to the old coach whichever roster the player now sits on, and freight arriving at the wrong destination is a materially worse outcome than a mismatched number. Two candidate resolutions: keep the refusal, or permit the move and state the freight consequence plainly on the confirmation. Recommend the latter, but it is a fulfillment decision rather than a roster one and should be confirmed with the Orders spec before changing.

To confirm before build: whether a move is permitted once the store is Active or Closed, and

whether moves are recorded in the store's activity history.

Metrics & Success Criteria

- Time from store-creation start to launch, and share of stores launching with no roster
- Share of League stores whose structure was built during creation vs. in this tab — a read on whether the creation step is carrying its weight
- Roster completeness at ordering-window close — committed players vs. estimated true team size
- Roster edit → storefront latency
- CSV import success rate; rows skipped per upload
- Manual add → save completion without duplicate-number errors
- **Duplicate-number errors surfaced at submit — target near zero** once numbers are locked at selection; a non-trivial rate means a number-entry surface is bypassing the picker
- **Share of rosters overriding the league-level default** — a read on whether per-roster scope was worth building, and on whether the bulk default is set to the right value
- **Personalization edits made against players who already have orders**, and the share that produce a divergence — the frequency of the case Epic 2 exists for
- **Share of divergences resolved through the pending-order update**, and the share discovered only after submission — if most are caught too late, the divergence line is not visible enough or the pending window is shorter in practice than it looks
- **Time from a divergent edit to the pending-order update** — how much of the pending window reps actually use
- **Rosters carrying grandfathered duplicates, and how long the condition persists** — a read on whether reps tidy it up or live with it, which is the direct test of the permissive policy
- **Roster Bank health** — pending captures remaining at window close, and one-click-commit share vs. manual split/merge
- Share of committed records carrying a contact channel — a read on how much reminder and share coverage the emergent path actually delivers
- Manager assignment completeness per division and team (freight-gate readiness for pending-order submission)
- Share-link and team-selector → order conversion
- Zero accidental clears/deletes (confirmation-gated); clear/delete actions tracked
- Structure-editor defects reported against only one of its two hosts (this tab, League creation Step 2) — target zero
- **Teams launching with zero assigned products** — the gap the amber state exists to prevent; expected to fall toward zero

- **Product assignments made from this tab vs. the Products tab** — a read on whether the structure-first surface matches how reps actually think
- **Drawer assignments by method** — click-Assign vs. drag, and division-header drops as a share of drags
- Product detail modal opens by entry point (hover card, thumbnail click, panel row title), and edits saved from the modal
- Time to answer "which of my teams is missing products", before and after the coverage table

---

Dependencies & Technical Constraints

Dependency & Technical Constraint

Requires genuine team-database integration: roster sync and member/role management. The **division / team / roster architecture** must be persisted as shared platform state — product assignment (Products tab), pending-order shipping containers (Orders tab), and the Settings manager mapping all read the same structure. **Team and roster are separate entities**: a team may exist with no roster, a roster attaches to exactly one team, and a team holds at most one roster. Division deletion must re-home teams as standalone rather than deleting them, carrying their rosters with them. **Clear Roster** must release all jersey numbers and preserve the roster record; **Delete Roster** must remove the roster entity while preserving its team; **team deletion** must remove the team from shared state, which product assignment and shipping containers consume. Neither clear nor delete may destroy pending Roster Bank captures or product assignments.

Order lines snapshot personalization; roster records do not feed them at read time. This is a constraint on the order model (owned by *Order Management*) that this tab depends on absolutely: jersey number, name on jersey, size, and shipping address are copied onto the order line at purchase, and no roster mutation may ever reach a placed order. Without the snapshot, every permissive behavior specified above becomes dangerous rather than convenient. The **divergence read** compares a player's current roster values against the snapshot on each of their order lines and is consumed by the Team Gear column and the gear drawer; it must be a derived comparison rather than a stored flag, so it resolves itself when either side changes. The **pending-order update** action rewrites one order line's snapshot from the current roster values and is available only while that order remains in the pre-submission pending state defined by *Order Management*, Epic B; it must be per line, with no bulk variant, and must be refused server-side once the order is submitted rather than merely hidden in the UI.

The **structure editor must be a single shared component** with two hosts — this tab and *Step 2 — Divisions & Teams (League Stores)* — parameterized to suppress branding, the

team code, and all Epic 6 product affordances in the creation context without forking behavior. Suppression is presentational only: team codes are still generated deterministically at creation, and division brand color still takes its default. Structure created in the wizard writes to this same shared state, committed atomically with the store on deploy.

Product visibility and assignment (Epic 6) requires the **catalog ↔ team assignment join read in the team→products direction**, not only product→teams as the Products tab reads it, with per-team and per-division aggregates driving the thumbnail strips, the division count pill, and the coverage table. Assignment scope must be modelled at **two levels — team and division** — so an inherited assignment can be rendered as a "via division" row and distinguished from a direct one; the inheritance rule for teams added to a division after a division-wide assignment is **open** (see Epic 6) and determines whether division scope is a stored rule or a drop-time fan-out. The **product card** and **product detail page** must be the **same components the Products tab uses**, rendered here as a floating card and a modal respectively — a second implementation would guarantee drift in card content, pricing display, and edit behavior. Saving from the modal writes the catalog record directly and must invalidate the strips, pills, cards, and coverage tables reading it. The **catalog drawer** requires catalog search, the three filter predicates (all / unassigned / not-on-scope, the last parameterized by the launching team or division), a launching-scope binding for click-Assign, and **drag-and-drop assignment** whose drop targets are team nodes and division headers on the canvas — writing the same team-code bindings the Products tab writes, with optimistic UI reconciled against the server result. Assignment permissions follow the Products tab's existing rules; this surface adds no new authority.

Manager mapping needs per-division fulfillment modes (league / division / individual teams), manager records (name, email, 10-digit phone, address), and backend **address verification** shared with the freight gate. Mapping state must be writable from both **Settings → Distribution & Fulfillment** and the division/team detail components without divergence.

Team codes are backend routing and reporting keys. They must be generated deterministically per division and team, stamped onto orders for container routing and reporting, and exposed read-only in the workspace. No rep-facing or shopper-facing code entry exists; shopper→team resolution happens through the storefront team selector (Roster-Optional Architecture, Epic D).

Duplicate-number policy must be persisted **per roster**, with the league-level Settings control implemented as a **bulk write** cascading through divisions to every roster plus a **mixed-state read** — not as a parent value that rosters inherit at read time, since a roster's own setting is authoritative once set. New rosters read the league-level default at creation only; that league-level field is the same one written by *Step 1 — Basics* Epic D1, with no separate creation-time storage. Number entry requires a **per-roster taken-number set** — the union of committed player records and pending Roster Bank captures for that team —

exposed to every selection surface (roster add row, edit drawer, storefront PDP, checkout) so unavailable numbers can be rendered disabled rather than validated after submit. The set must exclude the edited player's own current number and must invalidate on player delete, bulk delete, Clear Roster, capture commit, and capture dismissal. Validation remains as a server-side backstop for CSV import and direct field entry, and duplicate-number import errors must be evaluated against the target roster's policy rather than a league-wide rule.

Turning duplicates off must not trigger a retroactive validation pass. The policy write is a simple field update with no sweep, no conflict resolution step, and no mutation of existing records. The **grandfathered-duplicate read** is derived — a per-roster query for numbers held by more than one committed record while the policy is off — and drives the roster view's exception line only. It is a report, never a gate, and must resolve automatically when the underlying records change rather than requiring a dismissal.

Field-level validation must enforce the schema (string max-lengths, jersey integer **00–99**, valid email **when provided**, 10-digit phone **when provided**), with **First and Last Name as the only required fields** and no requirement that a record carry any contact channel. Surfaces that depend on a contact channel — Send Reminder, share-link sends, player-addressed system emails — must degrade to a disabled or copy-only state with a stated reason rather than blocking record creation.

The batch path needs a **CSV template generator** and a parse/validation engine returning **per-row / per-cell** error reporting with skip-and-import semantics (invalid rows skipped, valid rows imported) and an import preview; both the manual and CSV paths must be reachable from the League creation step as well as this tab. Deleting a record must release its jersey number back into the available pool. Sharing requires an email channel for the roster template and store-link sends, per-player invite-link generation, and an **associated-emails** join for recipient pools; the Team Gear column requires the same roster↔order join used by the My Team(s)-tab gear drawers.

The **Roster Bank** must be readable both per team and store-wide from this tab, with pending-capture counts joined onto division cards, team cards, and the stats header; capture storage, pre-grouping, and commit mechanics are specified in the Roster-Optional Architecture spec. Shared internal team services + database updates.